

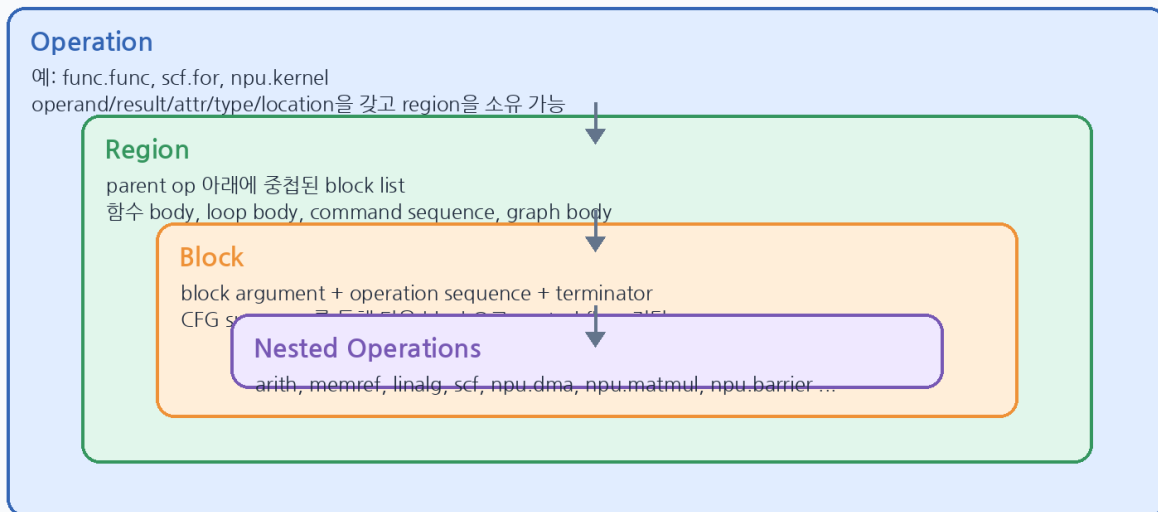
MLIR NPU Compiler

Lecture 03

Region 과 Block: NPU kernel / command region pseudo IR

AI 및 Edge/Embedded NPU Compiler 를 위한 MLIR 기초 강의자료

MLIR IR Nesting: Operation -> Region -> Block -> Operation



1. 강의 목표와 3 강의 위치

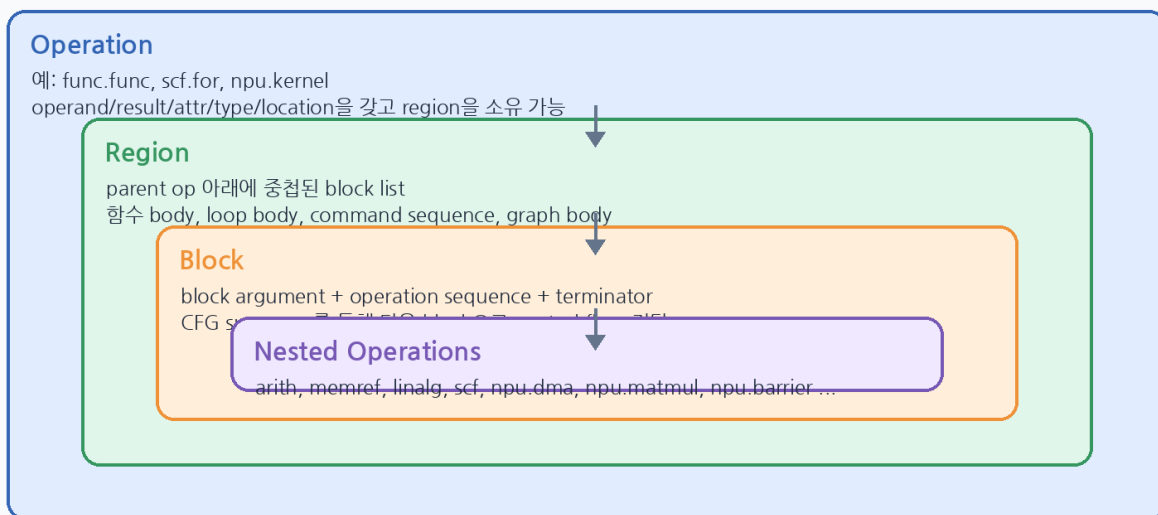
2 강에서 Operation, Value, Type, Attribute 를 읽는 법을 익혔다면, 3 강은 Operation 내부에 중첩되는 Region 과 Block 을 읽는 법을 다룹니다. NPU compiler 에서는 이 개념이 단순 문법을 넘어 kernel body, tiled loop, DMA command sequence, async token dependency, runtime command buffer ABI 로 이어집니다.

- Operation 이 0 개 이상의 Region 을 소유할 수 있음을 이해한다.
- Region 이 Block list 이고, Block 이 block argument 와 operation list 를 가진다는 구조를 읽는다.
- Terminator 와 successor 를 통해 SSACFG control flow 를 해석한다.
- LLVM PHI node 와 MLIR block argument 표현의 차이를 이해한다.
- NPU kernel/command region pseudo IR 을 설계하고 검증 checklist 를 작성한다.

2. 핵심 구조: Operation -> Region -> Block -> Operation

MLIR IR 은 재귀적으로 중첩됩니다. Operation 은 operand/result/attribute 뿐 아니라 region 을 가질 수 있고, region 은 block 들의 list 입니다. block 은 다시 operation list 를 담습니다. 이 구조 때문에 MLIR 은 함수, loop, graph, accelerator command sequence 를 하나의 IR framework 안에서 표현할 수 있습니다.

MLIR IR Nesting: Operation -> Region -> Block -> Operation



개념 모델

```

Operation
- operands / results / attributes / location / type
- regions[]
  Region
  - blocks[]
    Block
    - block arguments
    - operations[]
    - terminator / successors
  
```

3. Region: parent operation 이 의미를 정하는 semantic container

Region 은 자체적으로 하나의 고정 의미를 갖기보다 parent operation 의 semantics 에 의해 해석됩니다. 예를 들어 func.func 의 region 은 함수 body 이고, scf.for 의 region 은 loop body 이며, pseudo npu.kernel 의 region 은 kernel command body 가 될 수 있습니다.

Parent op	Region 의미	NPU compiler 해석
func.func	함수 body CFG	model-level kernel wrapper 또는 runtime entry
scf.for	loop body	tile loop, channel loop, batch loop
scf.if	then/else body	shape/runtime condition 에 따른 path
linalg.generic	structured op body	elementwise/computation semantics
npu.kernel (pseudo)	NPU kernel body	DMA/compute/barrier sequence 를 묶는 semantic unit
npu.command_buffer (pseudo)	command list	runtime ABI 로 serialize 할 대상

4. Block: argument, operation sequence, terminator

Block 은 region 내부의 기본 실행 단위입니다. SSACFG region 에서는 block 이 successor 를 가질 수 있으며, block 끝의 terminator 가 control flow 를 전달합니다. Block argument 는 함수 인자, loop induction variable, loop-carried variable, branch merge value 를 표현합니다.

Block argument 예시

```
^merge(%x : i32):
  // %x is a block argument.
  // Predecessor blocks pass a value when branching to ^merge.
  "use"(%x) : (i32) -> ()
  cf.br ^exit
```

NPU compiler 에서 block argument 는 tile index, loop-carried accumulator, async token, runtime descriptor 등을 안전하게 전달하는 수단이 될 수 있습니다. 단, custom dialect 설계 시 token 과 descriptor 를 operand/result 로 표현할지, region/block argument 로 표현할지 명확한 규칙이 필요합니다.

5. Terminator 와 successor

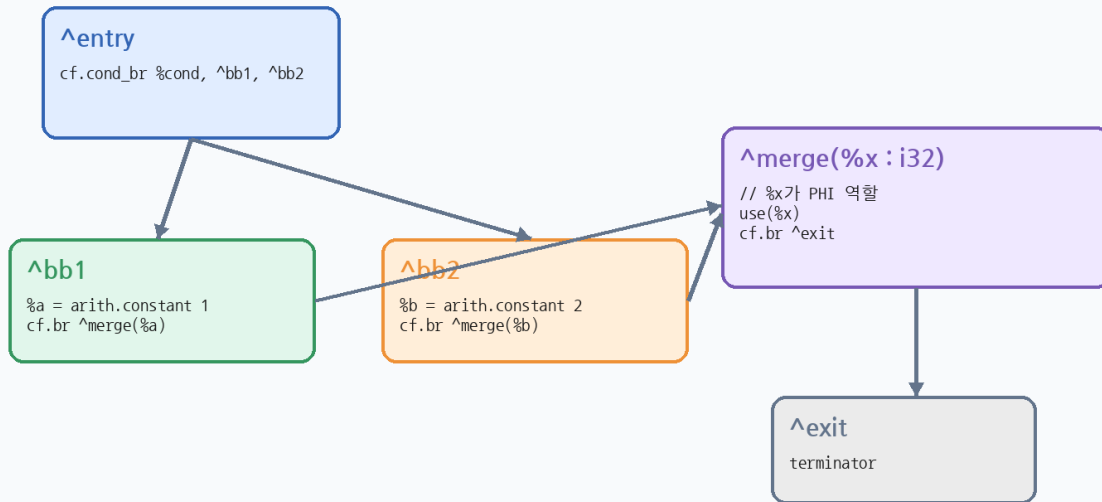
SSACFG region 의 block 마지막 operation 은 terminator 입니다. Terminator 는 다음 block 을 successor 로 지정하거나, parent operation 으로 값을 yield/return 합니다. Custom NPU dialect 를 설계할 때도 npu.yield, npu.return, npu.br 같은 terminator semantics 를 명확히 해야 verifier 와 lowering pass 가 안정적으로 동작합니다.

- func.func body 의 terminator: func.return 또는 return 계열 op
- cf dialect: cf.br, cf.cond_br 로 successor block 을 지정
- scf.for body: scf.yield 로 loop-carried 값을 다음 iteration/result 로 전달
- pseudo npu.command region: npu.yield 또는 npu.return 으로 command region 종료

6. Block argument vs PHI node

MLIR 은 LLVM IR 의 PHI node 대신 block argument 로 merge value 를 표현합니다. Predecessor terminator 가 successor block 으로 값을 넘기고, successor block 의 argument 가 그 값을 받습니다. 이 방식은 함수 argument, loop variable, branch merge 를 하나의 표현으로 통일합니다.

Block Arguments: MLIR의 PHI-node 대체 표현



핵심: predecessor가 successor block으로 값을 넘기고, successor block argument가 merge value가 됩니다.

Branch merge 예시

```

cf.cond_br %cond, ^then, ^else

^then:
  %a = arith.constant 1 : i32
  cf.br ^merge(%a : i32)

^else:
  %b = arith.constant 2 : i32
  cf.br ^merge(%b : i32)

^merge(%x : i32):
  // %x is the merged value.
  scf.yield %x : i32
  
```

7. Single-block region: scf.for

scf.for는 하나의 body region을 가지며, induction variable은 loop body region의 block argument로 표현됩니다. loop-carried variable이 있으면 iter_args 초기값이 추가 block argument로 매핑되고, scf.yield가 다음 iteration의 값을 넘깁니다.

loop-carried value

```

%sum = scf.for %i = %c0 to %c1024 step %c1
  iter_args(%acc = %zero) -> (f32) {
    %x = memref.load %A[%i] : memref<1024xf32>
    %next = arith.addf %acc, %x : f32
    scf.yield %next : f32
  }
return %sum : f32
  
```

NPU 관점에서는 %i가 tile index로 해석될 수 있고, %acc는 partial sum이나 status token 같은 loop-carried state로 해석될 수 있습니다. 나중에 tiling/scheduling pass는 이 region 구조를 이용해 DMA와 compute command를 재배치합니다.

8. Multi-block region: unstructured CFG 와 execute_region

MLIR 은 structured control flow 뿐 아니라 multi-block CFG region 도 표현할 수 있습니다. scf.execute_region 은 region 내부에 cf.cond_br/cf.br 를 사용한 block graph 를 담을 수 있습니다. 다만 accelerator command sequence 는 지나치게 unstructured CFG 로 만들면 scheduling 과 verification 이 어려워지므로, 가능한 structured form 에서 시작한 뒤 필요한 부분만 CFG 로 낮추는 설계가 유리합니다.

multi-block region

```
%r = scf.execute_region -> i32 {
  cf.cond_br %cond, ^then, ^else
^then:
  %a = arith.constant 1 : i32
  cf.br ^merge(%a : i32)
^else:
  %b = arith.constant 2 : i32
  cf.br ^merge(%b : i32)
^merge(%x : i32):
  scf.yield %x : i32
}
```

9. Dominance 와 value scope

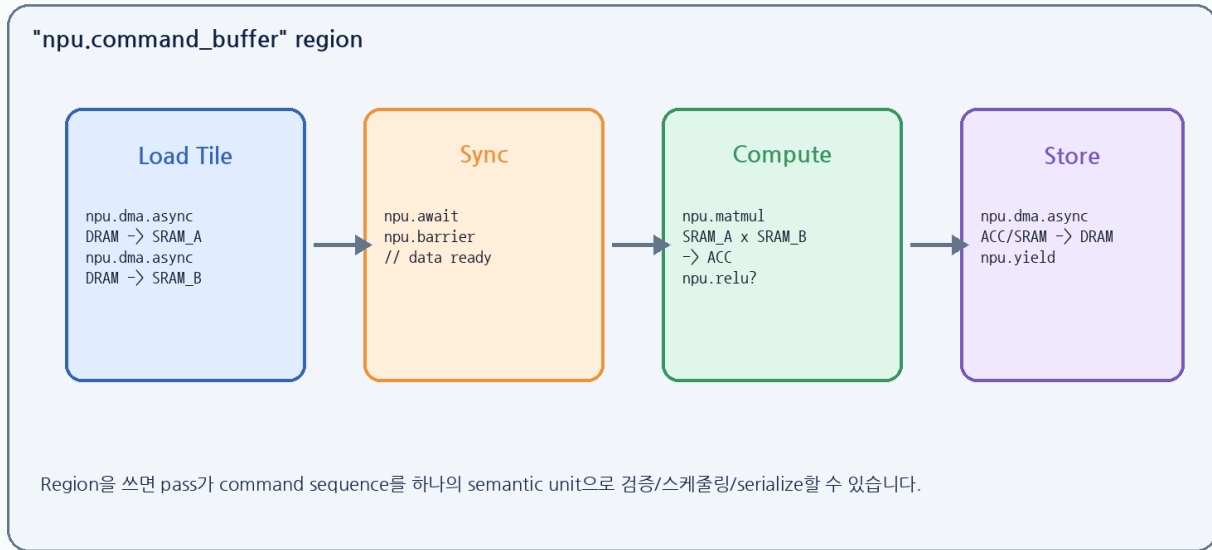
Region/block 을 다룰 때 가장 중요한 correctness rule 중 하나는 value scope 와 dominance 입니다. Region 내부에서 정의된 value 는 일반적으로 region 밖에서 임의로 사용할 수 없습니다. 또한 어떤 operation 이 value 를 사용하려면 그 value 가 해당 operation 에서 볼 수 있는 scope 에 있고 dominance 조건을 만족해야 합니다.

- Region argument 는 region 내부 깊은 operation 에서 사용할 수 있다.
- Region 내부에서 정의된 value 는 region 밖으로 불법 escape 하면 안 된다.
- Block argument type/count 는 predecessor terminator 가 넘기는 operand 와 맞아야 한다.
- Async token 은 dependency 를 표현하므로 token 사용 전후의 dominance/order 가 중요하다.

10. NPU kernel/command region 설계

NPU compiler 에서는 region 을 command sequence 의 semantic boundary 로 사용할 수 있습니다. 예를 들어 npu.command_buffer op 가 하나의 region 을 갖고, 그 region 안에 DMA load, await/barrier, matmul, requant, DMA store 를 순서대로 표현하면 pass 는 이 region 을 검증하고 runtime command buffer 로 serialize 할 수 있습니다.

NPU Command Region: DMA / Compute / Barrier를 Region으로 구조화



NPU command region pseudo IR

```

"npu.command_buffer"() {
  ^entry(%A: !npu.tensor_desc, %B: !npu.tensor_desc, %C: !npu.tensor_desc):
    %a_sram, %ta = "npu.dma.async"(%A) : (!npu.tensor_desc) -> (!npu.sram_view, !npu.token)
    %b_sram, %tb = "npu.dma.async"(%B) : (!npu.tensor_desc) -> (!npu.sram_view, !npu.token)
    "npu.await"(%ta, %tb) : (!npu.token, !npu.token) -> ()
    %acc = "npu.matmul"(%a_sram, %b_sram) : (!npu.sram_view, !npu.sram_view) -> !npu.acc_view
    %out = "npu.relu"(%acc) : (!npu.acc_view) -> !npu.sram_view
    %ts = "npu.dma.async"(%out, %C) : (!npu.sram_view, !npu.tensor_desc) -> !npu.token
    "npu.await"(%ts) : (!npu.token) -> ()
    "npu.yield"() : () -> ()
} : () -> ()

```

11. Verifier checklist for custom NPU region

Custom NPU dialect를 설계할 때 region/block invariants를 verifier로 강제해야 합니다. 그래야 이후 tiling, scheduling, bufferization, command serialization pass가 안전하게 동작합니다.

검사 항목	의미	실패 시 문제
Terminator	모든 block이 npu.yield/npu.return/cf.br 등으로 끝나는가	control flow 불명확
Block argument type	predecessor가 넘기는 operand와 type/count가 맞는가	merge value 오류
Token dependency	DMA token이 await/barrier 전에 소비되지 않는가	data hazard
Memory scope	SRAM/ACC view lifetime이 region 내부에서 닫히는가	SRAM leak 또는 alias 오류
Attribute vs operand	compile-time config와 runtime value가 구분되는가	schedule 재사용/캐싱 오류
Serialization order	command order가 deterministic한가	runtime command buffer mismatch

12. Pass 설계 관점

Region과 Block 구조를 잘 설계하면 pass가 단순해집니다. 예를 들어 NPU lowering pass는 npu.kernel region을 순회하면서 tile loop를 찾고, scheduler pass는 npu.command_buffer region 내부 command의 dependency

token graph 를 분석하며, runtime lowering pass 는 command region 을 linearized command buffer 로 변환합니다.

- Traversal: Operation 에서 시작해 nested region/block 을 재귀적으로 방문한다.
- Analysis: block argument, terminator successor, token def-use chain 을 추적한다.
- Transform: structured region 을 유지한 상태에서 fusion/tiling/scheduling 을 수행한다.
- Lowering: custom npu region 을 runtime ABI command descriptor 로 serialize 한다.

13. 실습 안내

1. examples/MLIR_NPU_Lecture_03_Region_Block_Examples.mlir 에서 region, block, block argument, terminator 를 색으로 표시한다고 가정하고 annotation 표를 작성한다.
2. scf.execute_region 예제에서 ^merge(%x : i32)의 %x 가 어떤 predecessor 값에서 오는지 def-use chain 을 설명한다.
3. MatMul tile 하나를 위한 npu.command_buffer pseudo IR 을 작성한다. DMA load, await, matmul, requant, DMA store, yield 를 포함해야 한다.
4. 작성한 pseudo IR 에 대해 verifier checklist 를 적용하고, 실패 가능한 case 를 3 개 이상 적는다.

14. 요약

- Region 은 parent operation 아래의 block list이며, parent op 가 semantics 를 정한다.
- Block 은 block argument 와 operation sequence 를 가지며, SSACFG 에서는 terminator 로 끝난다.
- MLIR 의 block argument 는 LLVM PHI node 역할을 대체한다.
- NPU compiler 에서 region/block 은 kernel body, command sequence, async token dependency, runtime command buffer ABI 설계와 직접 연결된다.
- 좋은 custom NPU dialect 는 region invariants 를 verifier 로 강제하고, pass 가 구조를 믿고 최적화할 수 있게 한다.

15. 참고자료

- MLIR Language Reference - <https://mlir.llvm.org/docs/LangRef/>
- Understanding the IR Structure - <https://mlir.llvm.org/docs/Tutorials/UnderstandingTheIRStructure/>
- MLIR Rationale: Block Arguments vs PHI nodes - <https://mlir.llvm.org/docs/Rationale/Rationale/>
- SCF Dialect - <https://mlir.llvm.org/docs/Dialects/SCFDialect/>